

PICK UP COUPONS

Company: Google

Round: Online Assessment

Year: 2021

Difficulty: Medium

Topic: Arrays, HashMap

Source: https://algo.monster/problems/google_oa_pick_up_coupons

Problem Description:

You are given a scratchcard containing a row of numbers. To win the game, you need to scratch at least two numbers that have the same value.

There is one important rule:

You can only scratch a continuous/consecutive part of the card. Each scratched number costs \$1. Our task is to find the minimum number of numbers we need to scratch to get two equal numbers.

If there are no two equal numbers, return : -1

Example

Consider:

[1, 3, 4, 2, 3, 4, 5]

The number 3 appears at positions 1 and 4.

To scratch both 3s, we must also scratch everything between them:

[3, 4, 2, 3]

There are 4 numbers, so the cost is:

4 dollars

Input Format:

The input is an array of non-negative integers.

Output Format:

Return the minimum number of consecutive elements that need to be scratched to contain two equal numbers. If no value occurs at least twice, return : -1

Sample Input:

arr[] = [1, 3, 4, 2, 3, 4, 5]

Sample Output:

Minimum Number of Consecutive Elements = 4

Explanation:

Let's understand the first example:

[1, 3, 4, 2, 3, 4, 5]

There are two 3s:

[3, 4, 2, 3]

To get both of them, we have to scratch all four numbers.

So the cost is:

4

There are also two 4s:

[4, 2, 3, 4]

This also costs 4.

Therefore, the minimum cost is:

4

For the second example:

[2, 5, 0, 3, 7]

Every number appears only once.

So it is impossible to get two equal numbers.

Therefore:

-1

Approach to Solve This Problem:

The main idea is very simple:

For every number, remember where we saw it last. We can use a HashMap for this.

Step 1: Start from the beginning

Go through the array one element at a time.

For example:

[1, 3, 4, 2, 3, 4, 5]

Step 2: Remember each number's position

When we see 1, we store:

1 → position 0

Then:

3 → position 1

4 → position 2

2 → position 3

Step 3: Find a repeated number

When we reach the second 3, we already know that 3 appeared at position 1.

The current position is 4.

So the consecutive part needed is:

$$4 - 1 + 1 = 4$$

Therefore, the cost is 4.

Step 4: Keep looking

We don't stop immediately because another pair might be closer.

For example:

[1, 2, 3, 1]

The two 1s require:

4 elements

But if we had:

[1, 2, 3, 2]

the two 2s require only:

3 elements

So we keep checking the whole array and store the smallest distance.

Java Code:

```
J Main.java > ...
1  import java.util.*;
2
3  public class Main {
4
5      public static int minCost(int[] arr) {
6
7          // Stores the last position of each number
8          HashMap<Integer, Integer> map = new HashMap<>();
9
10         int minCost = Integer.MAX_VALUE;
11
12         for (int i = 0; i < arr.length; i++) {
13
14             // If the number was already seen
15             if (map.containsKey(arr[i])) {
16
17                 int previousIndex = map.get(arr[i]);
18
19                 // Number of elements from previousIndex to i
20                 int cost = i - previousIndex + 1;
21
22                 minCost = Math.min(minCost, cost);
23             }
24
25             // Store/update the latest position
26             map.put(arr[i], i);
27         }
28
29         // No repeated number was found
30         if (minCost == Integer.MAX_VALUE) {
31             return -1;
32         }
33
34         return minCost;
35     }
36 }
```

```
36
37  Run | Debug
38  public static void main(String[] args) {
39
40      Scanner sc = new Scanner(System.in);
41
42      // Number of elements
43      int n = sc.nextInt();
44
45      int[] arr = new int[n];
46
47      // Input array
48      for (int i = 0; i < n; i++) {
49          arr[i] = sc.nextInt();
50      }
51
52      System.out.println(minCost(arr));
53
54      sc.close();
55  }
```

Solution:

We go through the array once and use a HashMap to remember where each number was last seen.

Whenever we see the same number again, we calculate:

Current index - Previous index + 1

This gives the number of consecutive elements we need to scratch.

We keep the smallest value found.

If no number appears twice, we return -1.

For:

[1, 3, 4, 2, 3, 4, 5]

the minimum cost is:

4

Understanding the Important Part of the Code:

1. HashMap

```
HashMap<Integer, Integer> map = new HashMap<>();
```

The HashMap stores:

number → last position

For example:

3 → 1

means that 3 was last seen at index 1.

2. Check if the number was seen

```
if (map.containsKey(arr[i]))
```

If the number is already in the HashMap, we have found a duplicate.

3. Find the cost

```
int previousIndex = map.get(arr[i]);
```

```
int cost = i - previousIndex + 1;
```

Suppose the same number appeared at index 1 and now appears at index 4.

Then:

$$4 - 1 + 1 = 4$$

So we need to scratch 4 numbers.

The +1 is important because both endpoints are included.

4. Keep the minimum

```
minCost = Math.min(minCost, cost);
```

There may be several repeated numbers.

We only want the smallest cost, so we keep updating minCost.

5. Store the latest position

```
map.put(arr[i], i);
```

We store the current position of the number.

Keeping the latest position is useful because a closer repeated occurrence will give us a smaller cost.

Complexity Analysis

Time Complexity: $O(n)$

We go through the array only once.

HashMap operations such as `containsKey()`, `get()`, and `put()` take $O(1)$ average time.

Therefore : Time Complexity = $O(n)$

Space Complexity: $O(n)$

In the worst case, every number is different, so the HashMap may store all n elements.

Space Complexity = $O(n)$